

Stack (DFS)

Mijn code begint met de startpositie op een stack te plaatsen. In de while-loop popt steeds de bovenste positie af. Als die positie het eindpositie is, stopt het. Anders dan kijkt het naar alle andere kanten via `maze.moves` (omhoog, omlaag, links en rechts). Elke positie die valid is, nog niet bezocht is, en een vrije cel 0 of eindpunt 2 is, wordt op de stack gegooid. Op hetzelfde moment slaat het op in de parent-dictionary vanuit welke cel die daarna bereikt werd.

Omdat het Stack is (LIFO), gaat het altijd verder op de laatste richting die hij bezocht, zo diep mogelijk, voordat hij terugkeert. Dat is de depth first search. Als het eindpunt gevonden is, loopt `ReconstructPath` terug via de parent-dictionary van eindpunt naar beginpunt, draait die om, en zet alles in de queue.

A*

De A* werkt anders: in plaats van een stack gebruikt hij een `SortedSet<Node>` die altijd gesorteerd is op F-waarde (laagste eerst). $F = G + H$, waarbij G het aantal stappen vanaf de start is, en H de Manhattan-afstand tot het eindpunt. Bij elke stap pakt hij de node met de laagste F (`openSet.Min`). Als dat het eindpunt is, stopt het. Anders gaat die naar eentje daarnaast. Voor elke daarnaast berekent hij een nieuwe G (current G + 1). Is die beter dan wat hij eerder had voor die cel, dan werkt hij de parent bij en voegt degene daarnaast opnieuw toe aan de open set met de nieuwe scores.

De `closedSet` zorgt ervoor dat al volledig verwerkte cellen niet opnieuw worden bekeken. De `NodeComparer` sorteert op F, dan G, dan invoegvolgorde, zodat bij delijke scores dezelfde keuze gemaakt wordt. (Net als bij DFS herstelt `ReconstructPath` aan het einde het pad via de parent-dictionary.)